الكلية متعددة التخصصات - ورزازات
FACULTÉ POLYDISCIPLINAIRE DE OUARZAZATE

Master IAA (Intelligence Artificielle et Applications)

Module : Machine Learning

TP : Prétraitement d'un Dataset en Machine Learning

Réalisé par :

Mohamed Ait Baha

Encadrant :

Prof , kamal oumari

Année universitaire : 2025 – 2026

Structure

1. Introduction
2. Partie 1 : Exploration
3. Partie 2 : Nettoyage
4. Partie 3 : Transformation
5. Partie 4 : Déséquilibre
6. Conclusion

Introduction

Dans le cadre du module Machine Learning, ce TP a pour objectif d'appliquer les techniques de prétraitement des données sur le dataset Titanic.

L'objectif principal est de préparer les données avant l'entraînement d'un modèle, en passant par plusieurs étapes : exploration, nettoyage, transformation et gestion du déséquilibre des classes.

Ces étapes sont essentielles pour améliorer la qualité des données et les performances des modèles de Machine Learning.

Partie 1 : Exploration

Dans cette première partie, nous avons effectué une exploration initiale du dataset Titanic afin de mieux comprendre sa structure et son contenu.

Nous avons commencé par afficher les premières lignes du dataset pour avoir une idée globale des données. Ensuite, nous avons identifié les types de variables (numériques et catégorielles) afin de savoir comment les traiter dans les étapes suivantes.

Enfin, nous avons vérifié la présence de valeurs manquantes dans chaque colonne, ce qui nous a permis de détecter les problèmes à corriger lors de la phase de nettoyage.

```python
# 1️⃣ Import pandas
# ============================
import pandas as pd
```

```python
# 2️⃣ Charger le dataset
# ============================
df = pd.read_csv("/content/drive/MyDrive/archive/tested.csv")
```

```python
# 3️⃣ Afficher les premières lignes
# ============================
df.head()
```

	PassengerId	Survived	Pclass	Name
0	892	0	3	Kelly, Mr. James
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)
2	894	0	2	Myles, Mr. Thomas Francis
3	895	0	3	Wirz, Mr. Albert
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)

```python
# 4 Types de données
# ============================
df.dtypes
```

	0
PassengerId	int64
Survived	int64

```python
# 5 Infos générales
# ============================
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  418 non-null    int64
 1   Survived     418 non-null    int64
```

```python
# 6 Valeurs manquantes
# ============================
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	86

Partie 2 : Nettoyage

Dans cette partie, nous avons procédé au nettoyage des données afin d'éliminer les incohérences et améliorer leur qualité.

Nous avons d'abord traité les valeurs manquantes. Les colonnes contenant un pourcentage élevé de valeurs manquantes ont été supprimées, tandis que les autres valeurs manquantes ont été remplacées par la moyenne pour les variables numériques et par le mode pour les variables catégorielles.

Ensuite, nous avons identifié les valeurs aberrantes à l'aide de boxplots. Pour les traiter, nous avons utilisé la méthode IQR, qui permet de détecter les valeurs extrêmes. Ces valeurs ont été soit remplacées par la médiane, soit supprimées, selon leur impact sur les données.

```
# 1️⃣ Pourcentage des valeurs manquantes
# ============================
mp = df.isnull().sum() / len(df) * 100
mp
```

...

	0
PassengerId	0.000000
Survived	0.000000
Pclass	0.000000
Name	0.000000
Sex	0.000000
Age	20.574163
SibSp	0.000000

```python
# 2️⃣ Supprimer colonnes > 40%
# ============================
sp = mp[mp > 40].index
df = df.drop(columns=sp)
```

```python
# 3️⃣ Remplacer valeurs manquantes numériques (mean)
# ============================
numériques = df.select_dtypes(include=['int64','float64']).columns
```

```python
for col in numériques:
    df[col] = df[col].fillna(df[col].mean())
```

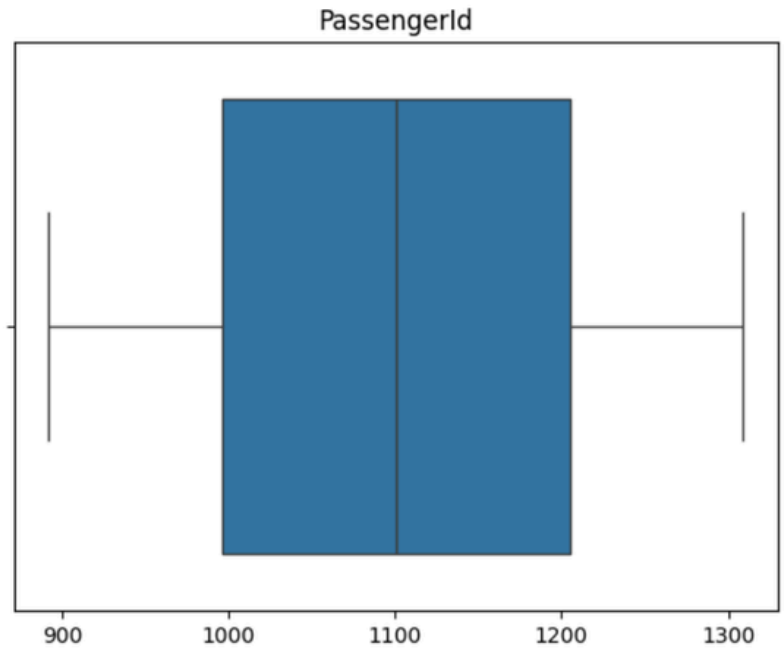
```python
# 4️⃣ Remplacer valeurs manquantes catégorielles (mode)
# ============================
categorical = df.select_dtypes(include=['object']).columns
```

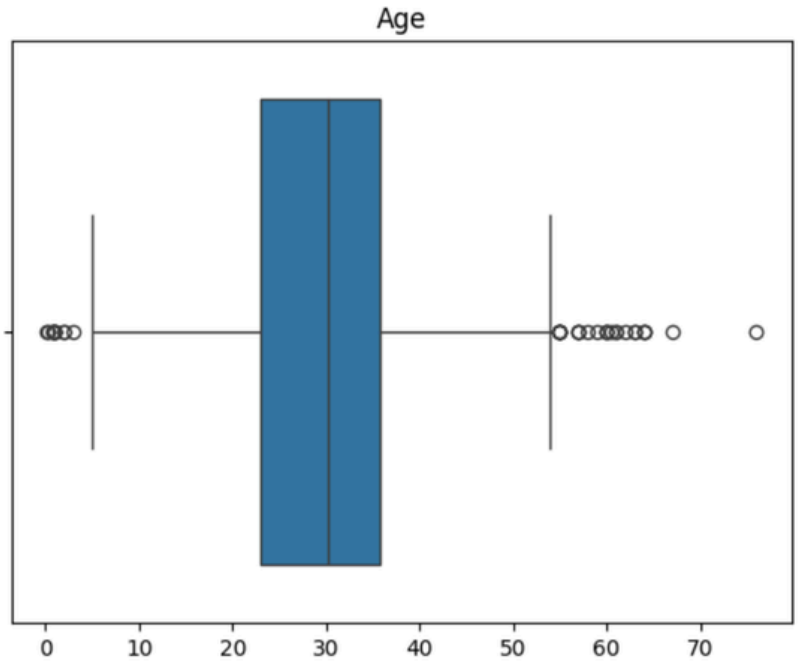
```python
for col in categorical:
    df[col] = df[col].fillna(df[col].mode()[0])
```

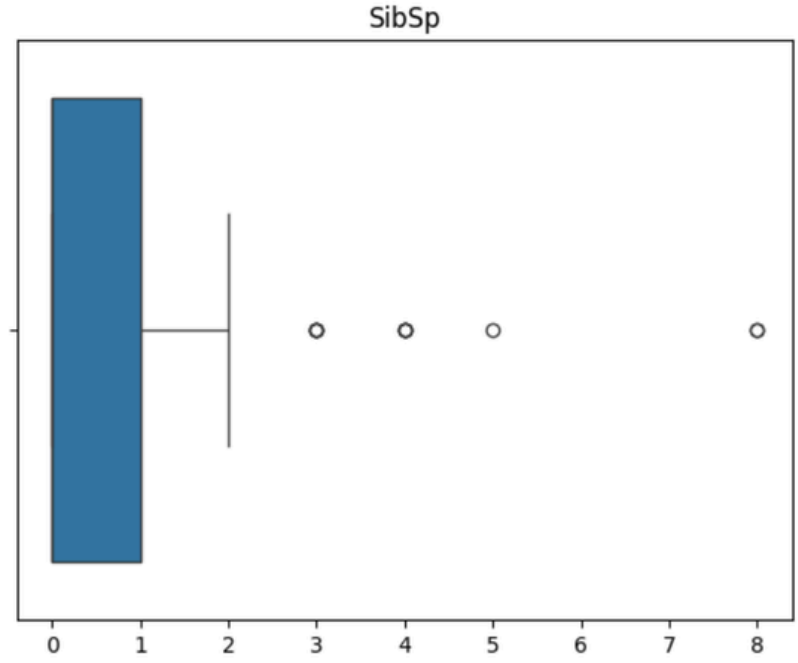
```python
import seaborn as sns
import matplotlib.pyplot as plt
```

```python
numeric_cols = df.select_dtypes(include=['int64','float64']).columns
```

```python
# 1️⃣ Boxplot (détection)
# ============================
for col in numeric_cols:
    plt.figure()
    sns.boxplot(x=df[col])
    plt.title(col)
    plt.show()
```


PassengerId
900
1000
1100
1200
1300


Age
0
10
20
30
40
50
60
70


SibSp
0
1
2
3
4
5
6
7
8

```python
# 2️⃣ Calcul IQR
# ============================
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    # 3️⃣ Remplacement par médiane
# ============================

    median = df[col].median()
    df.loc[(df[col] < lower) | (df[col] > upper), col] = median
```

```python
# 4️⃣ OU suppression des lignes
# ============================
df = df[(df[col] >= lower) & (df[col] <= upper)]
```

Partie 3 : Transformation

Dans cette partie, nous avons transformé les données afin de les rendre exploitables par les algorithmes de Machine Learning.

Nous avons commencé par encoder les variables catégorielles. Les variables nominales ont été transformées à l'aide de la méthode One-hot encoding, tandis que les variables ordinales ont été converties en valeurs numériques en respectant leur ordre.

Par la suite, nous avons appliqué des techniques de mise à l'échelle des données. La normalisation (Min-Max) a permis de ramener les valeurs dans un intervalle compris entre 0 et 1, tandis que la standardisation a permis d'obtenir des variables centrées avec une moyenne égale à 0 et un écart-type égal à 1.

Enfin, nous avons comparé les distributions des variables avant et après transformation à l'aide d'histogrammes afin de visualiser l'impact de ces transformations.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

```
# 1️⃣ categorical colonnes
# ============================
categorical_cols = df.select_dtypes(include=['object']).columns
print(categorical_cols)
```

```
Index(['Name', 'Sex', 'Ticket', 'Embarked'], dtype='object')
```

```
# 2️⃣ One-hot encoding (Sex, Embarked)
# ============================
df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True)
```

```python
# 3️⃣ Ordinal encoding (Pclass)
# ============================
df['Pclass'] = df['Pclass'].map({1:1, 2:2, 3:3})
```

```python
# 4️⃣ Variables numériques
# ============================
numeric_cols = df.select_dtypes(include=['int64','float64']).columns
```

```python
# 5️⃣ Normalisation (0 → 1)
# ============================
scaler_minmax = MinMaxScaler()
df[numeric_cols] = scaler_minmax.fit_transform(df[numeric_cols])
```

```python
# 6️⃣ Standardisation (mean=0, std=1)
# ============================
scaler_std = StandardScaler()

df_standard = df.copy()
df_standard[numeric_cols] = scaler_std.fit_transform(df_standard[numeric_cols])
```

Partie 4 : Déséquilibre

Dans cette dernière partie, nous avons étudié la distribution de la variable cible afin de vérifier s'il existe un déséquilibre entre les classes.

Après avoir identifié ce déséquilibre, nous avons appliqué différentes techniques pour le corriger. La méthode d'undersampling a été utilisée pour réduire la taille de la classe majoritaire, tandis que la méthode SMOTE a permis de générer de nouvelles observations pour la classe minoritaire.

Ces techniques permettent d'obtenir un dataset équilibré, ce qui améliore la performance et la fiabilité des modèles de Machine Learning.

```python
# 1️⃣ Vérifier déséquilibre
# ============================
df['Survived'].value_counts()
```

```
...
```

Survived	count
0.0	266
1.0	152

dtype: int64

```python
#◆ Undersampling

from sklearn.utils import resample

maj = df[df['Survived'] == 0]
mino = df[df['Survived'] == 1]

maj_down = resample(maj, replace=False, n_samples=len(mino), random_state=42)

df_under = pd.concat([maj_down, mino])

df_under['Survived'].value_counts()
```

...

Survived	count
0.0	152
1.0	152

```python
#◆ SMOTE (Oversampling)

from imblearn.over_sampling import SMOTE

X = df.drop('Survived', axis=1)
y = df['Survived']

sm = SMOTE(random_state=42)
X_res, y_res = sm.fit_resample(X, y)

print(pd.Series(y_res).value_counts())
```

...

Survived	count
0.0	266
1.0	266

dtype: int64

Conclusion

En conclusion, ce TP nous a permis de mettre en pratique les principales étapes du prétraitement des données.

Nous avons exploré le dataset, nettoyé les données en traitant les valeurs manquantes et aberrantes, transformé les variables et enfin corrigé le déséquilibre des classes.

Ces étapes sont indispensables pour garantir des résultats fiables et améliorer les performances des modèles de Machine Learning.